```
  module ClassMethods
    def first_level_class_method; 'ok'; end
  end

  include SecondLevelModule
end

class BaseClass
  include FirstLevelModule
end
```

BaseClass includes FirstLevelModule, which in turn includes SecondLevelModule. Both modules get in BaseClass's chain of ancestors, so you can call both modules' instance methods on an instance of BaseClass:

```
BaseClass.new.first_level_instance_method      # => "ok"
BaseClass.new.second_level_instance_method     # => "ok"
```

Thanks to include-and-extend, methods in FirstLevelModule::ClassMethods also become class methods on BaseClass:

```
BaseClass.first_level_class_method             # => "ok"
```

SecondLevelModule also uses include-and-extend, so you might expect methods in SecondLevelModule::ClassMethods to become class methods on BaseClass. However, the trick doesn't work in this case:

```
BaseClass.second_level_class_method            # => NoMethodError
```

Go through the code step by step, and you'll see where the problem is. When Ruby calls SecondLevelModule.included, the base parameter is not BaseClass, but FirstLevelModule. As a result, the methods in SecondLevelModule::ClassMethods become class methods on FirstLevelModule—which is not what we wanted.

Rails 2 did include a fix to this problem, but the fix wasn't pretty: instead of using include-and-extend in both the FirstLevelModule and the SecondLevelModule, Rails used it only in the FirstLevelModule. Then FirstLevelModule#included forced the includer to also include the SecondLevelModule, like this:

part2/chained_inclusions_fixed.rb

```
module FirstLevelModule
  def self.included(base)
    base.extend ClassMethods
    base.send :include, SecondLevelModule
  end

  # ...
```

Distressingly, the code above made the entire system less flexible; it forced Rails to distinguish first-level modules from other modules, and each module